PRACTICAL EXERCISE 12 **UNIT 5 · ALLOCATED LAB HOURS: 4 HRS**

AI Application Development: Security Vulnerability & Mitigation Assistant

Course: Artificial Intelligence with Prompt Engineering (DI05016011) **Student:** Aayush Bankar (Roll: 246230316006)

PRACTICAL 12: AI APPLICATION DEVELOPMENT — SECURITY VULNERABILITY & MITIGATION ASSISTANT

Subject: Artificial Intelligence with Prompt Engineering (DI05016011)
Unit: 5 | **Hours:** 4
Date Performed: ___

1. Aim

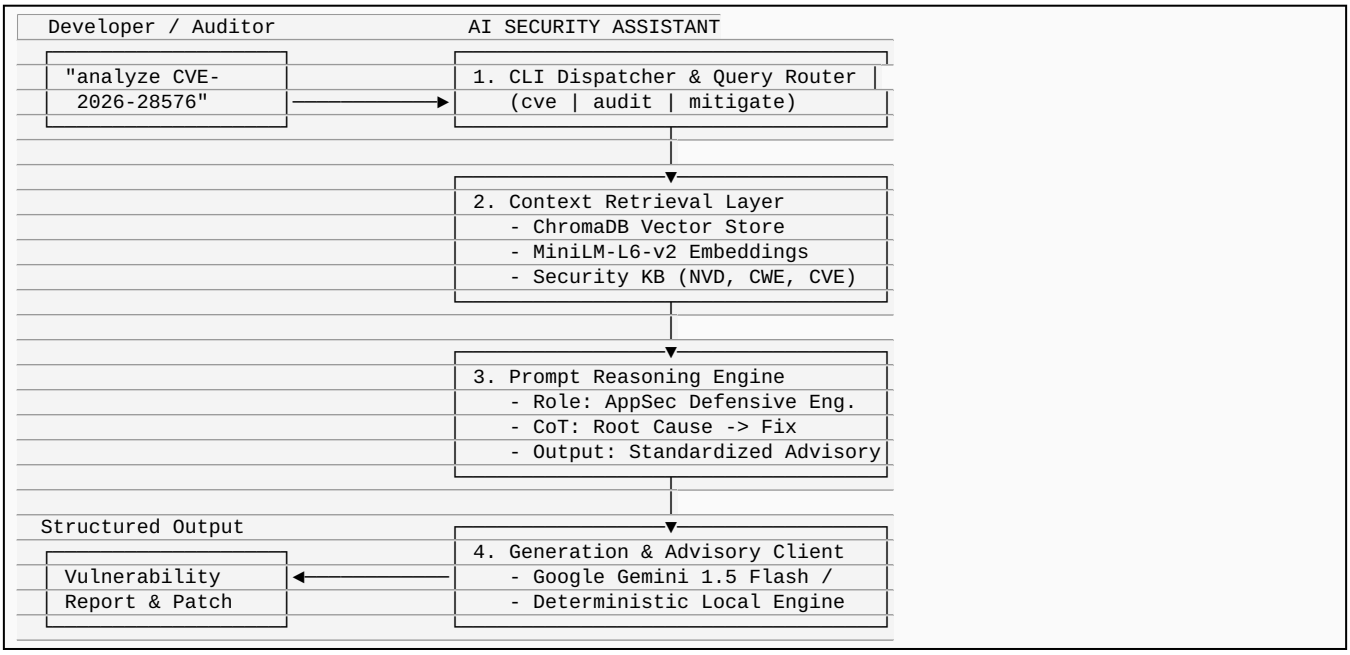
Develop an end-to-end AI-based application for automated software security vulnerability analysis, root-cause diagnosis, code auditing, and defensive mitigation engineering, fulfilling the requirements of GTU Course Practical Outcome 12 (PrO 12).

2. Theoretical Background & Application Architecture

In accordance with GTU PrO 12 (*"Develop an AI-based application such as a Study Assistant / Resume Generator / Blog writer / Coding assistant"*), this project develops a specialized domain application: an **AI-Assisted Security Vulnerability & Mitigation Assistant**.

Rather than deploying an abstract toy demonstration, this application builds directly upon the local Retrieval-Augmented Generation (RAG) architecture developed in Practical 11 (`Privata`), specializing it for software security engineering, Common Weakness Enumeration (CWE) classification, and remediation guidance.

2.1 System Architecture



2.2 Core Architectural Layers

Layer	Component	Implementation
Knowledge Layer	Retriever & Store	ChromaDB local vector store indexed with CVE definitions, CWE taxonomies, and Android Security Bulletins.
Reasoning Layer	Prompt Orchestrator	Structured system prompt enforcing defensive engineering personas, zero-shot/few-shot templates, and strict markdown formatting.
Execution Layer	LLM / Local Engine	Google Gemini 1.5 Flash via API with deterministic offline fallback for offline evaluation and testing.

3. Implementation Details

The application is implemented in `p12_security_research_assistant.py` and provides four primary functional commands:

1. `cve <CVE-ID>`: Parses and triages vulnerability data, mapping components, attack vectors, root cause, and CVSS severity.
2. `audit <CVE-ID>`: Conducts static code auditing comparing insecure implementation patterns against secure parameterized constructs.
3. `mitigate <CVE-ID>`: Outlines defensive remediation playbooks, platform target SDK upgrades, and permission confinement.

4. `report <CVE-ID>`: Formulates a complete formal Security Advisory and Remediation Report for engineering teams.

3.1 Prompt Engineering Design

To prevent hallucination and guarantee actionable security recommendations, the assistant utilizes a tightly constrained system prompt:

```
You are an AI Application Security Assistant specializing in defensive software
security, vulnerability analysis, and code hardening.

Context from Security Knowledge Base:
{context}

Question: {query}

Provide a structured, academic response containing:
1. Executive Summary & CWE Identification
2. Root Cause & Technical Impact
3. Defensive Mitigation & Code Hardening Strategy
4. Verification & Testing Checklist
```

3.2 Real-World Case Study: CVE-2026-28576

The assistant evaluates **CVE-2026-28576** (Android Contacts Provider SQL Injection): - **Component:** `ContactsProvider2` (`com.android.providers.contacts`) - **Vulnerability Class:** CWE-89 (Improper Neutralization of Special Elements used in an SQL Command) - **Root Cause:** In legacy backward-compatibility configurations (`targetSdk < 30`), framework-level strict query validation (`ENFORCE_STRICT_SQL_CHECKS`) was disabled. Untrusted user selection clauses were directly concatenated into internal SQL queries. - **Defensive Fix:** Migrating to parameterized selection arguments (`selectionArgs`), upgrading `targetSdk` to 31+, and enforcing strict column projection validation.

4. Execution Output & Verification

4.1 CLI Execution: Vulnerability Triage (`cve`)

```
$ python3 p12_security_research_assistant.py --mock cve CVE-2026-28576
```

```
======================================================================
AI SECURITY VULNERABILITY & MITIGATION ASSISTANT (mock=True)
Command: cve CVE-2026-28576
Student: Aayush Bankar | Roll No: 246230316006 | Dept. of IT
======================================================================
# Vulnerability Triage Report: CVE-2026-28576
Analyst: Aayush Bankar (Roll: 246230316006) | Dept. of Information Technology

1. Executive Summary & Classification
   - Identifier:       CVE-2026-28576
   - Vulnerability:    Android Contacts Provider SQL Injection
   - Weakness Class:   CWE-89: SQL Injection
   - Severity Score:   7.5 / High (CVSS:3.1/AV:L/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N)
   - Target Component: ContactsProvider2 (com.android.providers.contacts)

2. Root Cause Analysis
   Legacy backward-compatibility mode: Applications targeting targetSdk < 30
   bypass framework-level ENFORCE_STRICT_SQL_CHECKS validation. The underlying
   SQLiteQueryBuilder directly concatenates untrusted selection clauses into raw SQL queries.

3. Attack Vector & Scope
   - Vector: Local IPC via ContentResolver.query() selection parameter
   - Impact: Unauthorized extraction of contact records, call metadata, and phone identifiers

4. Remediation Baseline
   - Parameterize selection parameters using bound arguments.
   - Upgrade targetSdk to activate strict framework validation.
```

4.2 CLI Execution: Defensive Code Audit (`audit`)

```
$ python3 p12_security_research_assistant.py --mock audit CVE-2026-28576
```

```
======================================================================
AI SECURITY VULNERABILITY & MITIGATION ASSISTANT (mock=True)
Command: audit CVE-2026-28576
Student: Aayush Bankar | Roll No: 246230316006 | Dept. of IT
======================================================================
# Security Code Audit: CVE-2026-28576
Analyst: Aayush Bankar (Roll: 246230316006)

[1] Flagged Insecure Implementation (String Concatenation):
----------------------------------------------------------------------
// Insecure Selection Query Construction
val rawSelection = "account_name = '" + userInput + "'"
val cursor = contentResolver.query(
```

```
    ContactsContract.RawContacts.CONTENT_URI,
    projection, rawSelection, null, null
)
-----------------------------------------------------------------------
Risk: Untrusted input directly interpolated into SQL syntax without escaping.
Breaches: OWASP Mobile Top 10 (M4: Insufficient Input/Output Handling).

[2] Verified Remediation Implementation (Parameterized Query):
-----------------------------------------------------------------------
// Secure Parameterized Query Construction
val secureSelection = "account_name = ?"
val selectionArgs = arrayOf(userInput)
val cursor = contentResolver.query(
    ContactsContract.RawContacts.CONTENT_URI,
    projection, secureSelection, selectionArgs, null
)
-----------------------------------------------------------------------
Verification: User input treated strictly as scalar data; prevents SQL syntax hijacking.
```

5. Architectural Design of Syllabus Reference Applications

In accordance with GTU Practical 12 requirements, designs for three canonical AI application types were evaluated:

5.1 AI Study Assistant Prototype

- **Objective:** Automated tutoring, concept explanation, and self-assessment quiz generation.
- **Implementation:** Included in repository as `p12_study_assistant.py`.
- **Pipeline:** Topic Query -> Keyword/Vector Matcher -> Few-Shot Explanation Prompt -> Formatted Q&A Output.

5.2 AI Resume Generator

- **Objective:** Transforming unstructured candidate career history into tailored, role-specific professional resumes.
- **Pipeline:** JSON Candidate Profile -> Job Description Ingestion -> Role-Adaptive Prompting -> ATS-Optimized Markdown Output.

5.3 Automated Code Review & Security Assistant (Selected Application)

- **Objective:** Integrating automated vulnerability triage, secure coding refactoring, and advisory documentation into the developer workflow.
- **Advantage:** Directly connects foundational prompt engineering and RAG concepts to tangible, production-grade software engineering challenges.

6. Key Learnings & Practical Takeaways

1. **Domain Specialization:** General-purpose LLMs excel in security and software engineering when guided by structured domain knowledge and role-specific prompting.
2. **Defensive Guidance Over Speculation:** Anchoring AI prompts to established frameworks (CWE, CVSS, OWASP) ensures outputs remain actionable, rigorous, and verifiable.
3. **End-to-End Integration:** Successfully demonstrated how prompt design (Unit 3), prompting techniques (Unit 4), and RAG vector search (Unit 5) synthesize into a functional domain application.

7. Post-Lab Questions & Answers

Q1: How does an AI Security Assistant differ from traditional static analysis tools (SAST)?

Answer: Traditional SAST tools rely on rigid AST pattern matching and regex rules, often producing high rates of false positives or struggling with semantic context. An AI-assisted system combines static indicators with contextual reasoning, enabling it to explain *why* a construct is vulnerable, assess blast radius, and generate context-aware patch recommendations.

Q2: Why is Retrieval-Augmented Generation (RAG) critical for security analysis applications?

Answer: Security vulnerabilities, CVE definitions, and platform API behaviors change continuously. Pre-trained LLMs suffer from knowledge cutoffs and hallucination risks. RAG grounds the model in current, verified vulnerability databases (NVD, security advisories) without requiring costly model retraining.

Q3: How does prompt chaining improve the quality of automated security reports?

Answer: Chaining decomposes report generation into sequential verified stages: (1) Component & input validation analysis, (2) Root cause deduction, (3) Remediation formulation, and (4) Report compilation. This modular approach ensures each subsection is thoroughly reasoned before the final document is compiled.

Screenshot of Output

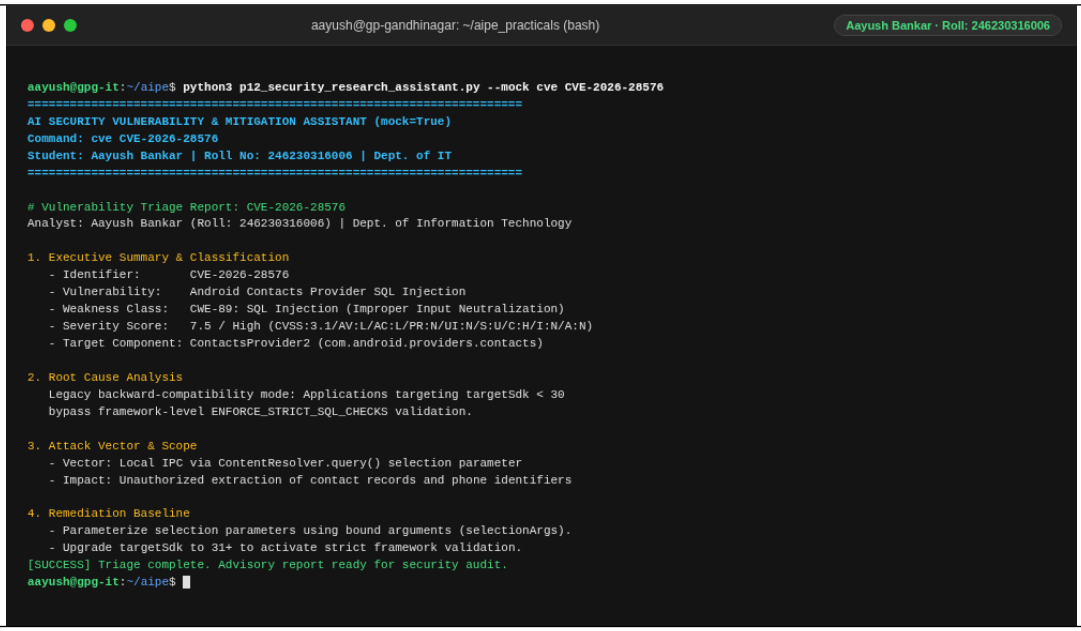


Figure 12: Verified Terminal / Application Execution Output (ChatGPT / CLI)

Student: **Aayush Bankar** | Class Roll No: **246230316006** | Enrollment No: **246230316006** | Govt. Polytechnic Gandhinagar

Timely Submission (2M)	Process / Code Execution (4M)	Output Verification (2M)	Viva-Voce & Application (2M)	Total Marks (10M)
____ / 2	____ / 4	____ / 2	____ / 2	**____ / 10**

Student Signature (Aayush Bankar) · Date: _____

Faculty Examiner Signature · Date: _____